
title: "IO SKY — Developer Handoff" author: "Manus AI"

date: "2026-06-03"

IO SKY — Developer Handoff

This document orients a new engineer to the IO SKY codebase: the stack, the directory layout, the build loop, the data model, the localisation system, the test strategy, and the safe places to extend. It is written to be read top-to-bottom once, then used as a reference. Pair it with

`IO_SKY_ULTRA_BLUEPRINT_LAUNCH_READY.md` (product overview),
`IO_SKY_STAGING_OPERATIONS.md` (runbook), `IO_SKY_DOMAIN_BINDING_HANDBOOK.md`
(domains), and `MANUS_INDEPENDENCE.md` (migration map).

1. Stack

The application is a single TypeScript project: **React 19 + Tailwind 4** on the client, **Express 4 + tRPC 11** on the server, **Drizzle ORM** over **MySQL/TiDB**, with **superjson** transport so Drizzle rows (including `Date`) flow end-to-end without manual serialisation. Authentication is Manus OAuth plus a local credential path and TOTP/SMS MFA. Server-side helpers provide LLM access, image generation, voice transcription, S3-compatible storage, and owner notifications. The frontend uses shadcn/ui components and a custom design system defined in `client/src/index.css`.

2. Directory Layout

The contract-first structure means most features touch four files. The directories an engineer edits are:

Path	Purpose
<code>drizzle/schema.ts</code>	Database tables and inferred types
<code>server/db.ts</code>	Query helpers returning raw Drizzle rows
<code>server/routers/*.ts</code>	tRPC procedures (feature-split)
<code>client/src/pages/*</code>	Page-level UI calling <code>trpc.*</code> hooks
<code>client/src/components/*</code>	Reusable UI and shadcn/ui
<code>client/src/lib/i18n/*</code>	Per-locale translation dictionaries
<code>shared/*</code>	Constants and types shared by client and server

Framework plumbing under `server/_core/` (OAuth, context, Vite bridge, LLM, storage, notifications) should be treated as infrastructure and left alone unless you are deliberately extending the platform.

3. The Build Loop

Adding or extending a feature follows a fixed five-step loop:

1. Update the schema in `drizzle/schema.ts`, then run `pnpm db:push` to generate and apply the migration.
2. Add a query helper in `server/db.ts` that returns raw rows.
3. Add or extend a procedure in `server/routers/<feature>.ts`, choosing `publicProcedure`, `protectedProcedure` or `adminProcedure`.
4. Wire the UI with `trpc.<feature>.useQuery` / `useMutation`, handling loading, empty and error states.
5. Cover the change with a Vitest spec in `server/*.test.ts` and run `pnpm test`.

There are no manual REST routes, no Axios client, and no hand-maintained shared contract files — the tRPC types are the contract.

4. Data Model (Selected)

The schema centres on organisations and users with role separation, the AI Scan domain, and the lead/CRM surface. Key tables include `users` (with an `admin | user` role enum), `organizations` and `organization_memberships`, `ai_scans` (including `reportToken` for public report access and `reportPdfKey` for the cached PDF), the lead/CRM tables that feed the admin “Top sources” view, and the MFA tables backing TOTP/SMS factors and recovery codes. Business timestamps are stored as UTC Unix milliseconds and converted to local time only at render.

5. AI Scan Subsystem

The questionnaire is defined in `shared/aiScanQuestionnaire.ts`; scoring and the model live in `shared/aiScanModel.ts` and the scoring module; the router is `server/routers/aiScans.ts`. Report access is token-gated via `reportToken`. The branded PDF is rendered server-side in `server/aiScanReportPdf.ts` using `pdfkit` (no browser dependency), uploaded with `storagePut`, cached against `ai_scans.reportPdfKey`, and returned as a signed URL by `aiScans.getReportPdf`. Question prompts and helpers are localised through the i18n dictionaries under the `aiscan.q.*` and `aiscan.start.*` key namespaces.

6. Localisation System

Ten locales are supported. English (`client/src/lib/i18n/en.ts`) is the authoritative superset; `translate()` resolves a key against the active locale, then English, then returns the key itself as a last resort. Because a missing key resolves to the key string (truthy), the `t("key") || "fallback"` idiom does **not** fall back — always ensure a key exists in all locales rather than relying on a code-level fallback. Parity is enforced by `server/i18n.completeness.test.ts` (every locale must cover all English keys) and a superset test (English must contain every key used anywhere). Transactional email strings live separately in `server/email-i18n.ts`.

When adding user-facing copy: add the English key first, then add translations to all nine other locales (scripts under `scripts/inject-*.mjs` show the idempotent injection pattern), and run the i18n tests.

7. Authentication & Authorisation

OAuth completes at `/api/oauth/callback` and sets a session cookie; context is built per request in `server/_core/context.ts`, exposing `ctx.user`. Use `protectedProcedure` for authenticated logic and `adminProcedure` for admin-only logic (it checks `ctx.user.role`). On the client, `useAuth()` exposes the current user and `getLoginUrl()` builds the login URL from `window.location.origin` — never hardcode domains, since frontend and backend run on separate origins. MFA is implemented through portal-agnostic `mfa.*` procedures and surfaced in both the Developer and Admin security centres.

8. Test Strategy

Tests are Vitest specs co-located under `server/*.test.ts`. The current suite is 364 tests across 28 files covering: workflow endpoints (bookings, contact, engineering), portal scoping (clientPortal, developer, viewAs), the AI Scan model/scoring/router and PDF rendering, MFA crypto and recovery-code flows, OAuth redirect, the staging gate, email localisation, and i18n completeness/parity. Treat writing or updating a test as a required step of every change, not an optional one. Run `pnpm tsc --noEmit` for types and `pnpm vitest run` for the suite before any checkpoint.

9. Where to Extend Safely

New product features should follow the build loop and live in their own router file under `server/routers/` (split once a router exceeds ~150 lines) with a matching page under `client/src/pages/`. New admin modules should reuse the `OperationalPage` shell and apply the sample-data badge if they are not backed by real data. New marketing copy must go through the i18n dictionaries. Files under `server/_core/` and the storage/LLM/notification helpers are the platform seams documented in `MANUS_INDEPENDENCE.md`; change them only when migrating providers.

10. Honesty & Integrity Conventions

The codebase deliberately avoids implying data, customers, certifications or guarantees that do not exist. Synthetic admin modules carry a “Sample data · module preview” badge; marketing claims are capability- or architecture-based rather than fabricated metrics; and placeholder CTAs are hidden rather than linked to `#`. Preserve these conventions when adding surfaces — if a number is not measured, do not present it as a fact.

11. Commands Quick Reference

Action	Command
Install deps	<code>pnpm install</code>
Dev server	managed by the platform; restart via tooling
Type check	<code>pnpm tsc --noEmit</code>
Run tests	<code>pnpm vitest run</code>
Apply schema	<code>pnpm db:push</code>
Format	<code>pnpm format</code>

This handoff reflects the codebase as verified on the authoring date: clean TypeScript compile and 364 passing tests.